SD7 — Control-Plane Work-Request API (async resource lifecycle)

> JD (`../0.req/oracle senior ic3.pdf`): "own … a bounded platform component or service, defining APIs and service contracts … versioning, interoperability, backward compatibility." This is the OCI-house pattern: every create/update/delete returns a **work request**. Cross-ref: `sd10` (the executor behind it), `sd13` (its main client), `../4.oci-architecture.md` §2 (CP vs DP). Tags: `[hot path]` · `[async]` · `[control plane]`. ❓ = follow-up + answer.

Index

0. Time budget
1. Why this is hard
2. Crux
3. Clarifying questions
4. FRs / NFRs
5. Back-of-envelope
6. High-level design
7. API
8. Data model
9. Deep dives (3)
10. Hygiene + ops
11. Pop-up questions
12. Trade-offs + defeaters
13. Wrap-up + card
14. References

---

§0 Time budget

Clarify 4 · why-hard 3 · FR/NFR 4 · HLD 8 · API+schema 6 (the API *is* the product here) · deep dives 10 · wrap 3.

§1 Why this is hard

```
naive: POST /instances blocks 90 s while the VM boots → 504s, retries, duplicate VMs
breaks at: any operation slower than a timeout; any client retry; any partial failure
```

- Naive: synchronous provisioning inside the HTTP request
- Breaks: gateway timeout ≪ provisioning time → client retries → **duplicate resources** and orphaned half-built state

- Minimum primitives: 202 + work-request handle · idempotency keys · a persisted state machine per operation · compensation for partial failure
- What the user wants: "kick it off, poll or get notified, retry safely, see why it failed"
- Cost of being wrong: orphaned billable resources; stuck resources in `CREATING` forever; automation (Terraform) that can't converge

🗣️ "The product here is the **contract**: async handle, idempotent verbs, monotonic states, and errors a machine can act on. Terraform is the real customer."

§2 Crux

- Exactly-once *resource effect* from at-least-once HTTP: idempotency key + state machine, not luck
- Every operation is: validate synchronously → persist intent → 202 → converge asynchronously → report terminal state
- States move one way (no un-failing); percent-complete is advisory, states are contractual
- Partial failure must end in a *terminal* state (SUCCEEDED / FAILED with cleanup), never limbo

§3 Clarifying questions

Question	Assumed
Which resources?	generic framework + one concrete (issue certificate)
Scale?	500 ops/s fleet-wide; op duration 1 s – 30 min
Clients?	console, SDKs, Terraform — machine clients dominate
Cancellation?	best-effort cancel where the executor supports it
Notification?	poll primary; events (webhook/stream) secondary

§4 FRs / NFRs

- FR: submit op → 202 + `work_request_id` · get status (+ per-resource entries, % complete, errors) · list by resource/tenant · cancel · idempotent submit
- NFR: submit p99 < 150 ms (validate + persist only) · status read 99.99% (it's what everyone polls) · state strongly consistent · audit every transition · per-tenant quotas + fairness (one tenant's storm can't starve others — `sd10 §10.3`)

§5 Back-of-envelope

- 500 ops/s × ~2 KB (request + entries + audit) ≈ 1 MB/s → ~86 GB/day raw; retain 90 d hot ≈ 2.5 TB — partition by month, archive
- Polling amplification: each op polled every 2–10 s for minutes → status reads ~50× submits → **status endpoint is the hot path**; serve from a read replica/cache with ≤1 s staleness, never the writer

§6 High-level design

```
[hot path]  Client ─► API GW (authz, throttle) ─► Resource API svc
                      validate + persist (resource row CREATING + work_request + outbox) ─► 202
[async]     Relay ─► queue ─► Workers (sd10 machinery: lease + fence) ─► call downstream / drive steps
                      each step: update wr entries + percent, append audit
[hot path]  Client ─ GET /work-requests/{id} ─► status read replica
[async]     Events: state transitions → stream/webhook (at-least-once, versioned payload)
[control plane] reaper: ops stuck in ACCEPTED/IN_PROGRESS past SLA → retry or FAIL + compensate
```

- Walkthrough — create certificate:
  1. `POST /certs` (Idempotency-Key K) → validate quota/authz → txn: cert row `CREATING` + wr `ACCEPTED` + outbox → 202
  2. Worker leases wr → `IN_PROGRESS` → HSM sign (sd13) → cert `ACTIVE`, wr `SUCCEEDED` (percent 100)
  3. Retry of step 1 with same K → same `work_request_id` returned, no second cert
  4. HSM fails permanently → wr `FAILED{code, message}`, cert row → `FAILED` (terminal, deletable) — no limbo

§7 API (the contract — spend time here)

```
POST /v1/certificates          Idempotency-Key: <uuid>   body: {...}
→ 202  {resource_id, work_request_id}     (Location: /v1/work-requests/{id})

GET /v1/work-requests/{id} →
{ "id": "...", "operation": "CREATE_CERTIFICATE",
  "status": "ACCEPTED|IN_PROGRESS|SUCCEEDED|FAILED|CANCELING|CANCELED",
  "percent_complete": 40,
  "resources": [{"type":"certificate","id":"...","action":"CREATED|UPDATED|DELETED"}],
  "errors": [{"code":"QuotaExceeded","message":"...","retryable":false}],
  "time_accepted": "...", "time_started": "...", "time_finished": "..." }

GET  /v1/work-requests?resource_id=&status=&cursor=      (list, cursor pagination)
POST /v1/work-requests/{id}:cancel  → 202 (best-effort)
```

- Contract rules to say out loud → states monotonic; clients may poll with exp backoff + jitter; `Retry-After` on 429 → errors: machine `code` + human `message` + `retryable` flag — Terraform branches on `code` → versioning: additive only in v1; new states never reuse old names; sunset headers before v2 → every mutating verb takes Idempotency-Key; GETs are pure

§8 Data model

```
CREATE TABLE work_requests (
  wr_id        uuid PRIMARY KEY,
  tenant_id    uuid NOT NULL,
  operation    text NOT NULL,
  status       text NOT NULL,              -- ACCEPTED|IN_PROGRESS|SUCCEEDED|FAILED|CANCELING|CANCELED
  percent      smallint NOT NULL DEFAULT 0,
  idem_key     text NOT NULL,
  fence        bigint NOT NULL DEFAULT 0,  -- sd10 lease/fence for the executing worker
  lease_until  timestamptz,
  accepted_at  timestamptz NOT NULL, started_at timestamptz, finished_at timestamptz,
  UNIQUE (tenant_id, idem_key)             -- idempotent submit
);
CREATE INDEX wr_by_status ON work_requests (status, accepted_at) WHERE status IN
('ACCEPTED','IN_PROGRESS');

CREATE TABLE wr_resources (
  wr_id uuid REFERENCES work_requests, resource_type text, resource_id uuid, action text,
  PRIMARY KEY (wr_id, resource_type, resource_id)
);
CREATE TABLE wr_errors (
  wr_id uuid REFERENCES work_requests, seq int, code text, message text, retryable boolean,
  PRIMARY KEY (wr_id, seq)
);
-- transitions audited to audit_log (append-only, as sd13)
```

§9 Deep dives

9.1 Idempotent submit (the follow-up you *will* get)

```
def submit(db, tenant, idem_key, operation, payload):
    """Same key -> same work request; retries create nothing."""
    with db.txn() as tx:
        existing = tx.one("""SELECT wr_id FROM work_requests
                             WHERE tenant_id=%s AND idem_key=%s""", (tenant, idem_key))
        if existing:
            return existing.wr_id, False            # replay: return original handle
        wr_id = uuid4()
        resource_id = create_resource_row(tx, tenant, operation, payload)  # state=CREATING
        tx.run("""INSERT INTO work_requests (wr_id, tenant_id, operation, status,
                  idem_key, accepted_at) VALUES (%s,%s,%s,'ACCEPTED',%s, now())""",
               (wr_id, tenant, operation, idem_key))
        tx.run("INSERT INTO outbox (wr_id) VALUES (%s)", (wr_id,))         # sd10 §10.2
        return wr_id, True
# Race: two racing submits with same key -> UNIQUE(tenant, idem_key) makes one fail
# -> catch unique violation, re-read, return the winner's wr_id. Say this.
```

```
@Transactional
public SubmitResult submit(UUID tenant, String idemKey, String op, JsonNode payload) {
    try {
        UUID wrId = UUID.randomUUID();
        UUID resourceId = resources.createPending(tenant, op, payload);
        repo.insertWorkRequest(wrId, tenant, op, Status.ACCEPTED, idemKey);
        outbox.enqueue(wrId);
        return new SubmitResult(wrId, resourceId, true);
    } catch (DuplicateKeyException e) {                    // (tenant, idem_key) unique
        UUID existing = repo.findByIdemKey(tenant, idemKey);
        return new SubmitResult(existing, repo.resourceOf(existing), false);
    }
}
```

9.2 State machine + compensation (no limbo, ever)

- Allowed transitions only: `ACCEPTED→IN_PROGRESS→{SUCCEEDED|FAILED}`; `*→CANCELING→CANCELED`; enforce with a conditional update (`WHERE status=$expected AND fence=$f`) — an illegal transition = 0 rows = bug surfaced, not absorbed
- Multi-step ops = a saga: each step has a compensator (created DNS record → delete it); on failure run compensators in reverse, then `FAILED` — resource ends terminal, deletable
- Reaper: `IN_PROGRESS` with expired lease → re-queue (fence bumps, sd10); `ACCEPTED` older than SLA → alarm (relay stuck)
- Cancel: flag checked between steps (cooperative); already-succeeded steps compensate or stand per op semantics — document per operation

9.3 Polling at scale + events

- Status reads ≫ writes: serve GET from read replicas; `ETag` / `If-None-Match` → 304 for unchanged (most polls); replica lag ≤ 1 s is fine — status is monotonic so a stale read is never *wrong*, only late. A client must never see SUCCEEDED then IN_PROGRESS → route a given wr's reads to one replica or gate on replica LSN
- Push: transition events to a stream (versioned envelope, at-least-once, consumers dedupe on `(wr_id, status)`); webhooks signed (HMAC) with retries + DLQ
- Quotas: in-flight wr per tenant (e.g., 100) → 429 + `Retry-After`; protects the executor fleet (fairness: sd10 §10.3)

§10 Hygiene + ops

- Metrics: submit p99, status p99, **age of oldest ACCEPTED** (relay health), % ops terminal < SLA, stuck-op count (page > 0 for 15 min), per-tenant in-flight
- Runbook: stuck IN_PROGRESS → lease expired? worker logs by wr_id (correlation id everywhere) → re-drive or fail+compensate manually via admin API (audited)
- Backward compat drill (JD): adding a new state = new minor version + capability flag; removing = never in v1

- Test: kill worker mid-saga → assert compensators ran and status FAILED; duplicate submit storm (same key ×100) → exactly one resource

§11 Pop-up questions

- *Why 202 and not sync for fast ops?* → "Uniformity is the product: Terraform writes one wait-loop for every resource. Offer fast-path completion by returning `SUCCEEDED` immediately in the wr body when the op finished inline — same contract, no special case."
- *Exactly-once?* → "At-least-once execution + idempotent steps + unique effect keys = exactly-once *effect*. Same argument as sd10; the wr row is the dedupe anchor."
- *Long-running (30 min) ops holding queue messages?* → "Message just carries wr_id; worker leases the row (visibility extended via heartbeat); progress persisted per step so a takeover resumes at the last completed step, not from zero."
- *percent_complete honest?* → "Advisory, monotonic, never 100 before terminal. Derive from completed saga steps, not time."
- *Why not workflow engines (Step Functions/Temporal)?* → "Buy them for the saga runtime if available — the *API contract* is still yours. Temporal replaces my worker+state persistence; it doesn't replace idempotency keys or the status endpoint."

§12 Trade-offs + defeaters

Decision	Chosen	Alternative	Flips when
Completion	poll + optional events	webhooks only	never — poll is the floor (firewalls, Terraform)
State store	Postgres (strong, single writer)	DynamoDB	multi-region active-active CP
Orchestration	own saga on sd10 machinery	Temporal/SFN	org already runs one
Fast ops	uniform 202	sync 200	pure reads (GETs are always sync)

- Defeaters → "Return 200 and finish in background without a handle" — the client can never distinguish lost from slow → "Idempotency via request-body hash" — two legit identical creates collide; the key is the client's *intent* id → "Percent from elapsed/expected time" — lies to automation; derive from steps

§13 Wrap-up + card

- Pitch: validate-persist-202; leased fenced executor; monotonic audited state machine with compensation; idempotency keys; status on replicas with ETag; events on top
- Why hard: HTTP retries × slow operations × partial failure — the contract must absorb all three
- 5 numbers: submit p99 150 ms · status reads 50× submits · in-flight quota 100/tenant · op SLA 30 min · retain 90 d
- 3 failures: duplicate submit (unique idem key) · worker death mid-saga (lease+fence+resume) · relay stall (oldest-ACCEPTED alarm)
- 3 defeaters: background-200 · body-hash idempotency · time-based percent
- Done when: contract recited (states, errors, versioning) in 60 s; submit txn written cold; saga compensation explained with the DNS example

§14 References

- OCI work-requests API docs (the pattern's origin for this team)
- AWS: async patterns in Builders' Library ("avoiding fallback", "timeouts & retries")
- Temporal docs — saga/compensation vocabulary
- Google AIP-151 (long-running operations) — the same contract at Google